

Speed Showdown: Fathom vs. Python

Empirical Head-to-Head Comparison on Apple Silicon M4 (10 Cores, Release LTO)

Empirical Benchmark Results: The Systems Difference

Benchmarks were executed offline using deterministic synthetic fixtures via `fathom bench` on macOS ARM64 (Apple M4, 10 cores, release build with LTO and stripped symbols).

TOOL LAYER OVERHEAD (4 LAYERS)

LAYER	ITERATIONS	TIME / CALL
registry.execute (Raw dispatch)	300	5,140 μ s
execute_batch (1 call)	300	7,614 μ s
execute_batch (8 calls amortized)	320	5,893 μ s (~0.75ms overhead)
ToolCall JSON Serde	100,000	752 ns

MULTI-THREADED PARALLEL SPEEDUPS

TASK	SEQUENTIAL	TOKIO SPAWN	SPEEDUP
16 $\times$ file_read (2MB each)	79.0 ms	25.8 ms	3.06 $\times$
8 $\times$ parse_html (1MB table)	130.3 ms	34.5 ms	3.78 $\times$
8 $\times$ code_symbols (240 files)	61.9 ms	20.4 ms	3.04 $\times$
web_feed (XML streaming)	—	—	1.11M items/s

Comparative Architecture Matrix

METRIC	PYTHON FRAMEWORKS (LANGCHAIN/AUTOGPT)	FATHOM (RUST COMPILED)
Cold Start Latency	2,500 – 8,000 ms	< 5 ms (500x Faster)
Tool Dispatch Overhead	25 – 150 ms	~0.75 ms (100x Faster)
Concurrency Model	Blocked by Python GIL	True Multi-Threaded Tokio Spawn